GO PROGRAMMING MASTER COURSE

Week 11, Day 1

Channels, pipelines, and the worker pool — the concurrency pattern to keep.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST WEEK

You can start Gs. Now make them talk safely.

- `go test -race` literacy; Create still sync.
- Cold-call: what is a channel? (pipe between goroutines.)
- Today: send/receive, close, range, `select`, pipeline, **worker pool**.
- Mutex deep-dive still week 12 — pool uses WaitGroup as glue.

TODAY

If Day 1 of week 11 works, you can

- Choose unbuffered vs buffered channels and say why.
- Close a channel and `range` until drained.
- Use `select` for multi-way wait (one live example).
- Sketch a pipeline (stage → stage).
- Implement a **worker pool** of N workers over a job channel.

SOFTWARE ENGINEERING

One pattern to remember: worker pool

Bound concurrency. N workers pull jobs. Results (or errors) come back on another channel. No unbounded go per item.

- SMS stretch: `POST /api/students/import` → `{created, failed}`.
- If the room struggles: hash-list / resizer **kata** only; sketch import on the board.

CHANNELS

Typed pipes between goroutines

```
ch := make(chan int)      // un
ch := make(chan int, 8)   // bu
ch <- 1                   // se
v := <-ch                 // re
```

- Unbuffered: send waits for receive (handshake).
- Buffered: send waits only when full.

DIRECTIONAL

Make intent obvious at the boundary

```
func produce(out chan<- int) {
func consume(in <-chan int) {
```

- chan<- send-only · <-chan receive-only.
- Compiler stops accidental reverse use.

CLOSE + RANGE

Sender closes · receivers drain

```
close(ch)
for v := range ch {
    // runs until ch is closed and
}
v, ok := <-ch // ok==false when cl
```

- Only the sender should close.
- Never send on a closed channel — panic.

SELECT

Wait on several channel ops

```
select {
case v := <-a:
    fmt.Println("a", v)
case b <- 1:
    fmt.Println("sent b"
case <-time.After(time.S
    fmt.Println("timeout
}
```

- One ready case runs; if several ready, pick at random.
- Default makes it non-blocking — use sparingly.

PIPELINE

Stage → stage → stage

```
nums := gen(1,2,3)
sq := square(nums)
for v := range sq { fmt.Pr
```

- Each stage: goroutine(s) + channel(s).
- Backpressure falls out of unbuffered/limited buffers.
- Worker pool is a special pipeline: many workers, one job queue.

WORKER POOL

The pattern to keep forever

```
jobs := make(chan Job)
results := make(chan Result)
for w := 0; w < N; w++ { go worke
// send jobs, close(jobs), collec
```

- Fixed N — no go per student/row.
- `WaitGroup`: wait until workers exit after `jobs` closed.

WAITGROUP · PREVIEW

Glue for “when are workers done?”

```
var wg sync.WaitGroup
wg.Add(N)
for i := 0; i < N; i++ {
    go func() { defer wg.Done(
}
wg.Wait()
close(results) // only after w
```

- Deep mutex/race week 12 — today WaitGroup is the join.

SMS STRETCH

POST /api/students/import

```
// body: [ {first_name, emai
// pool of N workers validat
// response: { "created": 3,
```

- Still in-memory store. Still no Postgres.
- Struggling room: kata only; sketch this API on the board.

KATA OPTIONS

If import is too much today

- **Hash list:** jobs are strings; workers compute SHA-256; collect digests.
- **Resizer sketch:** jobs are image paths (mock); workers “process”; count OK/fail.
- Same pool shape as import — swap the job type.

LIVE DEMO

CHANNEL BASICS

Send, receive, close, range

```
ch := make(chan string, 2)
ch ← "a"; ch ← "b"; close(ch)
for s := range ch { fmt.Println(s) }
```

LIVE DEMO

WORKER POOL

N workers · job channel · results

```
// starter/worker-lab
go test ./pool/
// or go run . — hash a list with 4 w
```

LIVE DEMO · STRETCH

IMPORT

Wire pool behind Gin

```
r.POST("/api/students/import", students.Import)
// bind []CreateStudentRequest → jobs → {created,
```

PITFALLS

Channel bugs that panic or hang

- Send on closed channel → panic.
- Forget to close → range hangs forever.
- Close from multiple goroutines → panic.
- Unbounded go per job → you’re not using a pool.

LAB · REST OF CLASS

Done when all of this is true

- Can explain unbuffered vs buffered
- Worker pool kata runs (hash list or equivalent)
- WaitGroup used to join workers before closing results
- No naked go per job in the solution
- Optional: import endpoint sketched or started

HOMEWORK

Import *or* documented kata

- **A:** `POST /api/students/import` with pool → `{created, failed}`.
- **B:** Documented worker pool under `internal/workers/` (delete before production if unused).
- PR explains N workers and who closes which channel.

NEXT SESSION

Week 12 — mutex, WaitGroup, context, races

- Fix the week-10 race for real.
- Request-scoped `context` on handlers.
- Bring a working pool mental model.

WORDS TO KEEP

Say these correctly next week

Channel

Typed pipe; send/receive synchronization.

select

Wait on multiple channel cases.

Pipeline

Stages connected by channels.

Worker pool

Fixed N workers pulling from a job queue.

LAYOUT REMINDER

Where lab code lives

```
docs/week-11-day-01/starter/
  worker-lab/                  # hash-list worker pool
  backend/
    internal/workers/          # homework option B
    internal/handlers/         # Import stretch
```

HONEST LIMITS

What this week is not

- Not mutex / race fix (week 12).
- Not Postgres bulk insert.
- Not unbounded fan-out disguised as a pool.
- Not every handler needs channels.

SAFETY

Still true

- Cap N and job list size in demos.
- No secrets in import fixtures — use example.com.
- `gofmt`; no binaries in git.

REVIEW BAR

Instructor merges when

- Pool uses fixed N + job channel (not go-per-item)
- Channels closed by the right owner
- PR explains created/failed or kata results
- Create/list still sync and green

Questions